From prompt to proven result

Architecture and task report for the interactive Cognitive AI Runtime reference scenario.

REFERENCE REQUEST

اعمل آلة حاسبة

"Make a calculator." The demonstration shows how a governed runtime clarifies the request, plans the work, checks the result, repairs a specific failure, and proves completion.

05	08	14	13
Lifecycle stages	Runtime modules	Data contracts	Evidence records

The task lifecycle

1. **Understand.** Capture the Arabic request and ask whether the calculator should run on the Web or Desktop.
2. **Plan.** Combine seven user requirements with one runtime quality requirement: safe invalid division. Record why the minimal stack was chosen.
3. **Work.** Create the calculator artifact while keeping agent status separate from task status. Completion still awaits verification.
4. **Check and repair.** The first candidate has 13/13 evidence records but only 7/8 acceptance checks: 8 divided by 0 produces Infinity. Apply a targeted zero-denominator guard.
5. **Result.** Re-run checks, reach 8/8, seal a verified result, and save a reusable experience under policy controls.

What the demo illustrates

The UI is an interactive reference simulation. Its evidence ledger records the failed check instead of treating evidence coverage as acceptance success.

How the runtime works

Eight modules coordinate the request, governance, execution, evidence, and completion gate.

Primary execution path

MOD-01 Intake > MOD-04 Analyze > MOD-05 Plan > MOD-06 Execute > MOD-08 Verify

MOD-02 / POLICY
Guardrails inform analysis and planning.

MOD-03 / EXPERIENCE
Trusted memory informs analysis; receives a verified pattern.

MOD-07 / EVIDENCE
Collects provenance and supplies the verifier.

REPAIR LOOP / MOD-08 -> MOD-06 / DIV-ZERO-01

Module responsibilities

Code	Module	Responsibility
MOD-01	Task intake	Capture the prompt and workspace context.
MOD-02	Policy engine	Apply security and quality rules.
MOD-03	Experience store	Read trusted patterns; save the verified result.
MOD-04	Requirement analyzer	Resolve Web vs Desktop; lock the task spec.
MOD-05	Planner and router	Choose the stack and record decisions.
MOD-06	Execution orchestrator	Build the artifact and run targeted repair.
MOD-07	Evidence ledger	Keep linked records, including failures.
MOD-08	Independent verifier	Gate completion on acceptance checks.

Architecture page interaction

The workspace map shows all 14 directed contracts. Users can filter contract types, select modules, inspect incoming and outgoing data, pause motion, and navigate the map on mobile.

What users can inspect

The three views keep the same calculator task in focus from request to evidence.

Workspace views

- Flow Studio - a five-stage journey with platform clarification, planning decisions, execution status, targeted repair, and a working calculator preview.
- Architecture - a connected, color-coded map of eight modules and 14 data contracts, plus an inspector for module relationships and policy outcomes.
- Evidence Ledger - traces requirements to decisions, module ownership, acceptance status, and provenance records.

Verification model

13 / 13	7 / 8	8 / 8
Provenance records captured	Checks before repair	Checks after repair

A recorded failure is still evidence. The independent completion gate blocks the first candidate because division by zero is unsafe. After the guard is applied, the simulated run shows zero unresolved required failures.

UI design delivered

- Responsive typography and navigation across desktop, tablet, and phone widths.
- A connected architecture diagram with subtle data-flow motion, a pause control, and reduced-motion support.
- Keyboard-accessible module and contract selection, visible focus states, and a mobile module picker.

PDF production workflow

This PDF was created through Taqrery's createReactPdfBlob render queue with its Cairo font registry. Report data was prepared from the ShiftCore source, rendered as a React PDF document, saved from the resulting blob, and visually checked after export.

Source: ShiftCore README, runtimeEngine.ts, App.tsx, RuntimeArchitectureDiagram.tsx, and the Taqrery PDF engine. Counts and scenario behavior are taken from the current reference implementation.